SENTINEL

BACKEND ARCHITECTURE REFERENCE

SENTINEL BACKEND GUIDE

The complete structural specification for the Sentinel FastAPI service — serving model, configuration, persistence, the API surface, connectivity and caching, the domain services, the SDR pipeline and its sidecars. Written so an engineer can reason about, extend or rebuild the backend from this document alone.

PRODUCT SENTINEL	SCOPE BACKEND / API	COMPILED 30 AUG 2026
SOURCE OF TRUTH BACKEND/	STACK FASTAPI · SQLALCHEMY 2.0 · SQLITE	RUNTIME PYTHON 3.12 · UVICORN

HOW TO READ THIS GUIDE

Sentinel's backend is a single FastAPI process. It is the only server: it terminates every API call, holds every WebSocket, owns the database, talks to every upstream and every radio, and serves the built Vue SPA as static files. There is no worker, no queue, no reverse proxy and no second service — the sidecar containers are decoders that call *in*, never components the app depends on to answer a request.

THE FIVE RULES THAT DEFINE THE BACKEND

RULE	WHAT IT MEANS IN PRACTICE
One process, one file	FastAPI serves API, WebSockets, map assets and the SPA from one uvicorn process backed by one SQLite file. Route order in `main.py` is load-bearing: the `/{full_path:path}` catch-all would swallow everything if it were not registered last. `main.py:166`
Degrade, never fail	An unreachable upstream is an expected state, not an error. The ADS-B proxy answers from cache and labels how (`X-Cache: HIT|MISS|RATED|THROTTLED|STALE|BYPASS`); TLE has a 12-hour stale window; the SDR broadcaster reconnects with capped backoff. A 503 is the last resort, not the first.
Preferences are rows, not columns	Everything configurable lives in one table as `(namespace, key, JSON value)` . Adding a setting is a seed entry in `default_config.json` , never a migration. `models.py:274`
Curated data lives in files	Band plans, SDR frequencies and satellite-radio metadata are git-tracked JSON under `backend/data/` . The file seeds the database on boot and is written back on edit, so the repo carries the reference data and the DB carries the runtime copy.
Validate at the edge, trust nothing	Every query goes through the ORM or bound parameters. Operator-supplied hosts are matched against a strict allow-list before reaching httpx; free-text names are stripped of control, zero-width and bidi characters; sidecar ingest is authenticated with a constant-time secret compare and fails closed.

SHAPE OF THE SERVICE

What ships in the process

Seven routers (100 HTTP endpoints, 4 WebSockets), 17 ORM models, 15 service modules, and an async lifespan that creates the schema, runs in-code migrations, seeds reference data and starts three classes of background task — the daily cleanup loop, one Sentry poller per enabled host, and one rtl_tcp broadcaster per connected radio.

What lives outside it

Four opt-in sidecar containers, all behind compose profiles and none required: `decoder` (dsd-fme digital voice), `aprs-decoder` (Direwolf), `adsb-decoder` (readsb), and the remote `rtl_tcp` daemons on the Sentry Raspberry Pis. Each connects to the backend, not the other way round.

> **NOTE ON FIDELITY** — every value, constant and behaviour in this guide was read out of the source, and is cited as `file:line` so it can be checked. Where the code is deliberately inconsistent, or a decision is recorded in an ADR, the guide says so rather than inventing a tidier answer.

CONTENTS

ONE PROCESS, ORDERED ROUTES

`backend/main.py` is 182 lines and does exactly three things: it defines the lifespan, includes the routers, and mounts the static surfaces in the one order that works. Everything else is delegated.

ROUTE ORDER — FIRST MATCH WINS

PATH	HANDLER	NOTES
`/api/**`	7 routers	Included first, so no API path can be captured by the SPA catch-all. `main.py:123–129`
`/ws/sdr/{id}`, `…/iq`, `…/decode`, `…/decode/audio`	sdr router	The four WebSockets are declared on the SDR router itself, which carries no prefix so it can own both `/api/sdr/*` and `/ws/*`. `routers/sdr.py:54`
`/health`	inline	`{status, timestamp}` — the container/orchestrator probe. `main.py:133`
`/favicon.ico`	inline	Served from `frontend/assets/` before the mounts. `main.py:139`
`/assets/**`	StaticFiles	Map tiles, PMTiles archives, sprites, glyph fonts, favicons. `main.py:149`
`/fonts/**`, `/spa-assets/**`	StaticFiles	The hashed Vue bundle. Vite is configured with `assetsDir='spa-assets'` precisely so it cannot collide with the map `/assets` mount above. Both mounts are conditional on the directory existing. `main.py:155–160`
`/{full_path:path}`	serve_spa	Returns the SPA `index.html` so vue-router can handle client-side routes — or a 503 telling you to build the SPA if the bundle is absent. `main.py:166`

> **WHY INDEX.HTML IS NEVER CACHED** — the catch-all sets `Cache-Control: no-cache, no-store, must-revalidate`. `index.html` references hash-named JS/CSS, so a cached copy pins the browser to a bundle that no longer exists after a rebuild. The hashed assets themselves stay immutably cacheable. `main.py:170–177`

WHAT THE ORDERING BUYS YOU

Because the routers are included before any mount and the catch-all is registered last, adding a router can never shadow a static path and adding a static path can never shadow an API route — the one ordering constraint a contributor has to remember is that the catch-all stays at the bottom of `main.py`. Both SPA mounts are conditional on their directories existing, so a checkout with no built bundle still boots and every API endpoint still answers; only the browser UI is missing, and the catch-all says so explicitly with a 503 rather than a 404.

THE LIFE OF ONE REQUEST

Every API call takes the same six steps. Nothing in the backend deviates from this shape — which is why a new endpoint is almost always a small addition rather than a new mechanism.

1. **Route match** — FastAPI matches the path against the routers in inclusion order. No match, and the catch-all serves the SPA instead.
2. **Validation** — path and query parameters are coerced to their annotated types and the body is parsed into its Pydantic model. A bad value ends here as a 422, before any handler code runs.
3. **Dependencies** — `Depends(get_db)` opens an `AsyncSession` from the shared factory. It closes when the response is finished, not when the handler returns.
4. **Handler** — reads settings through `db_helpers`, delegates real work to a service, and returns a `JSONResponse`. Error decorators wrap the whole call, so a service raising `RuntimeError` becomes a 503 without the handler catching anything.
5. **Background tasks** — anything that must not delay the response (flight-history recording, above all) is queued as a `BackgroundTask` with its own session.
6. **Response** — headers carry the diagnostics: `X-Cache` on ADS-B, `Cache-Control` on the SPA entry. Nothing else is added globally; there is no middleware stack.

STARTUP, SHUTDOWN & THE SIGNAL CHAIN

STARTUP SEQUENCE

The `lifespan` async context manager runs this order on every boot. Each step is idempotent — a fresh install and a five-year-old database both converge on the same state.

1. **create_tables()** — `Base.metadata.create_all`, then a list of guarded `ALTER TABLE` statements, data backfills and `CREATE INDEX IF NOT EXISTS`. Also creates the recordings directory inside the data volume. `database.py:50`
2. **migrate_sdr_radios_to_settings()** — one-time lift of the legacy `sdr_radios` table into `UserSettings(sdr.radios)`; returns immediately if the key already exists. `database.py:186`
3. **prune_removed_settings()** — deletes rows belonging to withdrawn features *before* the seeders run, so a stale key can never be mistaken for a live default. Currently drops `sdr.trunkTrackingEnabled` and `sdr.channel_maps` (ADR-0004). `database.py:254–287`
4. **seed_default_settings()** — renames two legacy air keys, deletes nine obsolete ones, then inserts anything from `default_config.json` that has no row yet. Existing rows are never overwritten. `database.py:471`
5. **seed_sdr_data_from_files()** and **seed_sdr_bandplan_from_file()** — seed groups, frequencies, search ranges and the band plan from `backend/data/*.json` when the tables are empty, then mirror live state back into the files. `database.py:391, 430`
6. **backfill_satellite_radio_store()** — merges `satellite_radio.json` over the persistent store; the file wins for keys it defines, UI-only entries survive. No-ops when already in sync. `database.py:443`
7. **resolve_ingest_secret()** — materialises the decoder ingest secret into the shared volume so the sidecar can authenticate. `services/sdr_decode.py:58`
8. **resume_persisted_aprs()** — best-effort restart of background APRS decode on the persisted radio; a missing radio or dead dongle is logged and skipped, never blocking startup. `routers/sdr.py:1508`
9. **_daily_cleanup_loop()** task + **fleet_poller.start_all()** — flight-history and APRS-station cleanup runs once immediately, then every 24 h; one poller task starts per enabled Sentry host. `main.py:37, 71`

SHUTDOWN — AND THE SIGNAL CHAIN THAT MAKES IT WORK

SIGTERM and SIGINT are *chained*, not replaced: the handler wakes every SDR subscriber queue and decoder first, then calls uvicorn's original handler. Without the pre-wake, uvicorn waits on WebSocket tasks blocked on a queue that will never be fed, and `--reload` deadlocks. On the way out the original handlers are restored, the cleanup task is cancelled *and awaited*, and the pollers, decoders and broadcasters are shut down in that order. `main.py:73–109`

> **DO NOT REMOVE THE CHAIN** — this is the single most load-bearing piece of process plumbing in the backend. Replacing the handler instead of chaining it hangs shutdown; skipping the `await cleanup_task` hands uvicorn a still-cancelling task and produces the same symptom.

SETTINGS, ENV & THE COMPOSE WIRING

`backend/config.py` is a single Pydantic `BaseSettings` class read from the environment and an optional `.env`, exported as one module-level `settings` singleton that every other module imports. There is no second config mechanism and no runtime reload — changing one of these values means restarting the process. User-facing preferences are a different system entirely (section 04).

EVERY SETTING, WITH ITS DEFAULT

SETTING	DEFAULT	WHY THIS VALUE
`db_path`	`backend/sentinel.db`	Overridden to `/app/data/sentinel.db` in compose so the DB lives in a named volume.
`adsb_ttl_ms`	10 000	Freshness window — matches the upstream's own update cadence.
`adsb_stale_ms`	60 000	How long a dead upstream can still be papered over with old data.
`adsb_upstream_base`	`https://api.adsb.lol/v2`	Keyless drop-in for airplanes.live, which closed its v2 endpoint behind an auth key.
`adsb_min_request_interval_ms`	5 000	Floor on outbound spacing per upstream host — these feeds ban faster clients.
`adsb_rate_limit_max_wait_ms`	5 000	Capped at one interval so a burst of callers cannot build an unbounded queue.
`adsb_rate_limit_penalty_ms`	60 000	Cooldown after a 429. adsb.lol's limits are "dynamic based on environment load", so the fixed interval is a floor, not a guarantee.
`adsb_rate_limit_max_penalty_ms`	600 000	Ceiling on an honoured `Retry-After`, so a hostile header cannot park the feed for hours.
`tle_ttl_ms` / `tle_stale_ms`	6 h / 12 h	TLEs change slowly; Celestrak publishes daily.
`tle_manual_ttl_ms`	30 d	The user typed it in — don't expire it on them.
`celestrak_iss_url`	Celestrak `GROUP=active`	Seeded into `space.onlineUrl`.

HOW A VALUE REACHES THE PROCESS

Precedence is Pydantic's: **environment variable → `.env` file → class default**. Names map case-insensitively, so `DB_PATH` sets `db_path`. Compose supplies `DB_PATH`, `DECODER_*` and `APRS_DECODER_PCM_PORT` directly, and reads `SENTINEL_DECODER_SECRET` from the developer's own `.env` when pinning a secret.

Two config values are also seeded *into the database* as user-facing defaults on first boot — `adsb_upstream_base` becomes `air.onlineDataSourceURL` and `celestrak_iss_url` becomes `space.onlineUrl`. After that the database row wins: the env value only ever populates an absent row. `database.py:173–183`

```
# backend/config.py:105
class Config:
    env_file = ".env"
    env_file_encoding = "utf-8"

# one singleton, imported everywhere
settings = Settings()
```

SIDECAR & FLEET SETTINGS

DECODE, APRS AND SENTRY

SETTING	DEFAULT	WHY THIS VALUE
`decoder_pcm_port`	7355	Backend serves 48 kHz mono s16 PCM here; dsd-fme connects *in* (SDR++ TCP-audio-sink convention).
`decoder_audio_udp_port`	7356	Decoded voice comes back over UDP.
`decoder_ingest_secret`	*empty*	Empty means auto-generate. Set it only to pin an explicit secret; it then takes precedence over the file.
`decoder_secret_file`	`/run/decoder/secret`	Shared volume both containers mount.
`decoder_default_bw_hz`	12 500	DMR/P25/NXDN are all ~12.5 kHz channels.
`sdr_relay_control_port_offset`	2	IQ 1234 → control 1236. Must match the relay's `RELAY_CONTROL_PORT`.
`sdr_relay_control_timeout_s`	2.0	Past this, the claim is treated as "not owner" and the channel as absent.
`aprs_decoder_pcm_port`	7357	Distinct from 7355 so voice and APRS can decode concurrently on two dongles.
`aprs_decoder_default_bw_hz`	15 000	2 m APRS is ~15 kHz narrowband FM — wider than voice so the 1200/2200 Hz AFSK tones pass cleanly.
`aprs_station_ttl_ms`	300 000	Fallback retention for a heard station; the `land/aprsRetentionMinutes` preference overrides it.
`sentry_poll_interval_s`	2.0	Per-host status poll.
`sentry_poll_backoff_start_s` / `_max_s`	2.0 / 30.0	Exponential on consecutive failures, capped so a long-dead host is still retried rather than abandoned.
`sentry_connect_timeout_s` / `_read_timeout_s`	3.0 / 5.0	The Pi may be slow or simply off the network.

WHAT COMPOSE ACTUALLY SETS

Only a handful of environment variables are passed to the app container; everything else runs on the class defaults above. The decoder containers get their own set, pointing back at these same ports.

VARIABLE	VALUE AND PURPOSE
`DB_PATH`	`/app/data/sentinel.db` — moves the database into the `sentinel_db` named volume so it survives a rebuild.
`PYTHONUNBUFFERED`	`1` — so log lines reach `docker logs` as they happen rather than in blocks.
`DECODER_INGEST_SECRET`	`${SENTINEL_DECODER_SECRET:-}` — empty by default, which is exactly what selects the auto-generated shared-volume secret.
`DECODER_PCM_PORT`, `DECODER_AUDIO_UDP_PORT`	`7355` / `7356` — restated in compose so the app and the decoder service can be pointed at the same numbers in one place.
`APRS_DECODER_PCM_PORT`	`7357` — used only when the `aprs` profile runs.

> **PORTS DIFFER INSIDE AND OUT** — uvicorn listens on **8000** inside the container and compose publishes it as **8080** on the host. Vite's dev proxy expects **8080**, which is why the dev loop runs the dockerised backend rather than a bare `uvicorn` on its own default port.

ENGINE, PRAGMAS & SCHEMA EVOLUTION

SQLite via `aiosqlite`, driven by async SQLAlchemy 2.0. One file, one engine, one session factory. There is no Alembic: the schema is built from the ORM models at startup and evolved by a list of guarded `ALTER TABLE` statements in `create_tables()`.

ENGINE AND CONNECTION PRAGMAS

SQLite serialises writes on a single file lock, and Sentinel writes concurrently — several ADS-B cache upserts plus a background flight-history writer will collide and raise `database is locked`. Three pragmas are set on every connection through a `connect` event listener: `database.py:27–33`

PRAGMA	EFFECT
`journal_mode=WAL`	Readers proceed during writes instead of blocking.
`busy_timeout=5000`	A new writer waits up to 5 s for the lock rather than failing instantly.
`synchronous=NORMAL`	Fewer fsyncs; acceptable durability for a cache-and-preferences store.

`check_same_thread=False` is required for async use, and `sessionmaker(expire_on_commit=False)` keeps ORM objects usable after commit without a second query. Requests get a session from the `get_db` dependency; background tasks and services that run outside a request open their own `AsyncSessionLocal()`. `database.py:16–41, 147`

> **LOCKS STILL LEAK THROUGH** — `upsert_setting()` retries an `OperationalError` three times with a 0.2 s-per-attempt backoff, because a small preference write can still lose the race to a busy background writer and would otherwise surface as a 500. The ADS-B proxy takes the other route: on a locked commit it rolls back and serves the fresh upstream data anyway, labelled `X-Cache: BYPASS`. `db_helpers.py:57–95 · routers/air.py:180–186`

SCHEMA EVOLUTION WITHOUT A MIGRATION TOOL

`create_tables()` runs four kinds of statement, all idempotent, all inside one `engine.begin()`:

A **create_all** — creates any table that does not exist, from the ORM metadata. It will *not* add a column to an existing table, which is the entire reason B exists.

B **22 guarded ALTER TABLE statements** — each wrapped in `try/except OperationalError: pass`, so "duplicate column" is the success case on a database that already has it. This is the migration mechanism. `database.py:63–91`

C **Backfills** — per-mode default bandwidths for frequencies created before the column existed (WFM 500 k, NFM 12.5 k, AM 10 k, USB/LSB 3 k, CW 500), a default sample rate, slugs for groups predating the `slug` column, and an `INSERT OR IGNORE` that lifts the legacy single `group_id` into the many-to-many link table. All conditioned on `WHERE … IS NULL`, so re-running changes nothing. `database.py:96–133`

D **Five indexes** via `CREATE INDEX IF NOT EXISTS`, all serving flight-history reads: registration lookup, `(flight_id, ts)` snapshot ranges, `last_seen DESC`, snapshot `ts`, and the flight time window. `database.py:134–141`

> **THE RULE FOR ADDING A COLUMN** — add it to the model *and* append an `ALTER TABLE` line to the list in `create_tables()`. A model-only change works on a fresh database and silently breaks every existing one; the failure surfaces later as an `OperationalError: no such column` at query time, not at boot.

FILE ↔ DATABASE RECONCILIATION

Four curated JSON files under `backend/data/` are git-tracked reference data, and each follows the same contract: **the file seeds an empty table, the database is authoritative at runtime, and every edit writes back to the file.** `json_store.py` centralises the read (fail-soft) and atomic write, and writes deliberately fail soft — returning `False` and logging — so a read-only filesystem (a baked image with no bind mount) still runs. Satellite radio is the one variant: the file is merged *over* the store on every boot, so a hand-edit takes effect at the next restart while UI edits survive for satellites the file does not mention.

`services/json_store.py · database.py:443`

THE 17 MODELS & THE SETTINGS STORE

Seventeen tables, and they are not all the same kind of thing. Two are pure caches that may be deleted at any time (`adsb_cache`, `tle_cache`); four are append-mostly records the user can prune (`air_flights`, `air_snapshots`, `sdr_recordings`, `aprs_stations`); the rest are registries and configuration that only an explicit action removes.

MODEL / TABLE	DOMAIN	PURPOSE & KEY DETAIL
`AdsbCache`	Air	One row per bounding-box query, keyed `"{lat:.4f}_{lon:.4f}_{radius}"`. Holds the raw upstream JSON plus `fetched_at` / `expires_at`.
`AirMessage`	Air	Notification feed (emergency squawks, system alerts). `msg_id` is client-generated and unique — creates are idempotent. Dismissal is a soft-delete flag.
`AirTracking`	Air	Aircraft the user selected, one row per ICAO hex, with a camera-follow flag.
`AirAircraft`	Air	Identity registry keyed by registration (`r`), falling back to ICAO hex. Persists across sessions so history accumulates.
`AirFlight`	Air	A continuous flight session. A new row opens when an aircraft reappears after `FLIGHT_GAP_MS` (10 min).
`AirSnapshot`	Air	Position time series — lat/lon/alt/gs/track/baro_rate/squawk at one instant.
`TleCache`	Space	Raw 3-line element text keyed by NORAD ID, with a `source` of `online|url|upload|manual`. Manual entries get a far-future expiry and are never auto-refreshed.
`SatelliteCatalogue`	Space	Permanent identity per NORAD ID — name, category, category source, plus the radio columns (uplink/downlink/beacon/CTCSS/transponder/status/notes). Never deleted by a TLE import, only by an explicit clear.
`SdrRadio`	SDR	Legacy table, superseded by `UserSettings(sdr.radios)`; retained because the startup migration reads it.
`SdrFrequencyGroup`	SDR	Named group with a rename-stable `slug` and a colour (default `#c8ff00`).
`SdrStoredFrequency`	SDR	A bookmark carrying its full tuning state: mode, squelch, gain, bandwidth, sample rate, volume, zoom, waterfall min/max, scannable and favourite flags.
`SdrFrequencyGroupLink`	SDR	Junction table — a frequency may belong to several groups.
`SdrSearchRange`	SDR	Low/high/step sweep definition with a stop-on-signal threshold and dwell.
`SdrRecording`	SDR	One captured clip; `status` is `recording|complete|error`, with optional companion IQ file.
`SentryHost`	SDR	How to reach one Sentry Pi plus its bearer token and poll telemetry. Unique on `(address, port)`. Device state is deliberately *not* stored here (ADR-0009).
`AprsStation`	Land	Latest fix per callsign, upserted on each position packet; swept when older than the retention window.
`UserSettings`	All	The preferences store — see below. Unique on `(namespace, key)`.

THE PREFERENCES STORE

USERSETTINGS — ONE TABLE, EVERY PREFERENCE

Rows are `(namespace, key, JSON-encoded value, updated_at)` with a uniqueness constraint on the pair. Namespaces are `app`, `air`, `space`, `sea`, `land` and `sdr`. Values are arbitrary JSON — booleans, strings, objects, whole arrays of radios.

Three helpers in `db_helpers.py` are the whole access surface: `get_setting_row`, `get_setting` (parsed, with a default, swallowing decode errors) and `upsert_setting` (encodes, commits, retries on lock). Routers, the database module and several services all go through these rather than hand-writing the select.

The store is also what the Application Config editor exports and imports: `GET /api/settings/config/preview` renders every row as a nested JSON document, and `POST /api/settings/config/upload` replaces settings from an uploaded copy.

Seeding & hygiene

`default_config.json` defines the first-boot defaults — 4 `app` keys, 7 `air`, 5 `space`, 2 `sea`, 5 `land` and 12 `sdr`. Keys beginning `_` (such as `_comment`) are ignored by the parser.

Because rows outlive the features that created them, two lists keep the store honest: `_REMOVED_SETTING_KEYS` (deleted every boot, for withdrawn features) and `_OBSOLETE_KEYS` (renames and pre-`offgrid` spellings). Without them a dead key keeps appearing in the config editor and in exported JSON, indistinguishable from a live setting.

> **ONE DELIBERATE EXCEPTION** — `sdr.groups` is never purged. It is the live `{name, slug}` catalogue owned by the groups table; deleting it each boot would drop user renames until the next mutation re-synced it. `database.py:505–512`

CONFIG IMPORT IS NOT A BLIND WRITE

An uploaded config is rejected whole if it is not a JSON object, or if `app.location` is an invalid or partial coordinate pair — validated up front so a bad value cannot be half-persisted. Missing `sdr.radios` ids are assigned sequentially. Keys that have moved out of the config into dedicated data files (SDR frequencies, groups, search ranges, band plan, satellite radio) are skipped on both export and import, so an old exported config cannot silently overwrite the dedicated stores. `routers/settings.py:340–428`

WHERE EACH KIND OF DATA LIVES

Three stores, with different lifetimes and different rules about who wins on a conflict. Knowing which one a value belongs in is most of the decision when adding something new.

STORE	HOLDS	LIFETIME & AUTHORITY
SQLite `sentinel.db`	Caches, history, SDR catalogue, Sentry hosts, every user preference.	Survives restarts and rebuilds (it lives in a named volume). Authoritative at runtime for everything it holds.
JSON files `backend/data/`	SDR frequencies, groups, search ranges, the band plan, satellite-radio metadata.	Git-tracked. Seeds an empty table on first boot and is rewritten on every edit, so the repo carries the curated reference data. Writes fail soft on a read-only filesystem.
Process memory	rtl_tcp connections and broadcasters, decode bridges, Sentry device snapshots, rate-limiter slots, the ingest secret.	Lost on restart, deliberately. None of it is state the user owns — it is all either re-derivable or live device state that belongs to whatever produced it.

> **THE ONE ASYMMETRY WORTH REMEMBERING** — for SDR data the database wins and the file follows; for satellite radio the file wins and the store follows, merged on every boot. That is why hand-editing `satellite_radio.json` works and hand-editing `sdr_frequencies.json` gets overwritten by the next sync.

SEVEN ROUTERS, ONE HUNDRED ENDPOINTS

One router per domain or resource, each aggregated in `main.py` with its own prefix and tag. No router holds unrelated concerns; when one grows a second concern it is split. FastAPI's own docs are moved out of the way at `/api/docs`, `/api/redoc` and `/api/openapi.json` so they cannot clash with the SPA. `main.py:112–129`

ROUTER INVENTORY

ROUTER	PREFIX	ROUTES	OWNS
`air`	`/api/air`	14	ADS-B proxy, notification messages, tracking list, flight history and replay windows.
`space`	`/api/space`	16	ISS and arbitrary-satellite position/track/footprint, pass prediction, day/night terminator, the whole TLE lifecycle, satellite-radio file.
`land`	`/api/land`	1	APRS stations heard by the Land decoder. Sea is scaffolding with no router at all.
`settings`	`/api/settings`	6	Namespace/key CRUD plus config preview and upload.
`sdr`	*none*	39 + 4 WS	Radios, groups, frequencies, search ranges, recordings, connect/disconnect/status, the data files, decode and APRS control and ingest — and all four WebSockets. Carries no prefix precisely so it can own both `/api/sdr/*` and `/ws/*`.
`sentry`	`/api/sdr/sentry-hosts`	19	Sentry host CRUD, health probe, remote device management and the Wi-Fi hotspot surface.
`adsb_source`	`/api/sdr/adsb`	5	Resolving, claiming and tuning the Sentry dongle that feeds Off Grid ADS-B (ADR-0003), plus the config the readsb sidecar polls.

CONVENTIONS EVERY ENDPOINT FOLLOWS

CONVENTION	DETAIL
Request bodies are Pydantic	Declared as a `BaseModel` at the top of the router file, with the endpoint it serves named in the docstring. FastAPI's validation produces the 422s.
Sessions arrive by dependency	`db: AsyncSession = Depends(get_db)`. Never a module-level session; never a session created inside the handler unless it outlives the request (background tasks and WebSockets open their own).
Status codes are explicit	`status_code=201` on creates, `204` on deletes that return nothing, `200` where a body comes back.
Errors are decorated, not repeated	`@handle_service_errors` maps `RuntimeError → 503` and anything else → 500; `@handle_unexpected_errors` maps everything → 500. Both preserve the exact JSON shape the hand-written blocks used, which is why they could replace ~10 identical try/except blocks in the space router without changing a single response. `error_handlers.py:19, 42`
Route order inside a router matters	The settings router registers `/config/preview` and `/config/upload` *before* `/{namespace}`, or the parameterised route would shadow them. `routers/settings.py:434`
Docstrings are the contract	Every router module opens with its endpoint list; every handler documents its own semantics including error codes. The WebSocket handlers document the full message protocol in both directions.

SHARED HELPERS

`utils.py`

`slugify()` — Unicode-aware and NFKC-normalising. It folds Latin accents but preserves other scripts: `Café Naïve → cafe-naive`, `Москва радио → москва-радио`. Returns an empty string when nothing slug-able remains, and callers fall back to `group-{id}`.

`clean_group_name()` — validates free text: it strips C0/C1 controls, zero-width and bidi-override characters, collapses whitespace runs, and rejects empty or over-60-character names with `InvalidGroupName` so the API can answer 422.

`cache.py`

Three functions, twenty lines, used by every cache in the codebase: `now_ms()`, `is_fresh(expires_at)`, and `is_within_stale(fetched_at, stale_ms)`. Time is Unix milliseconds everywhere in the backend — every timestamp column, every cache field, every WebSocket frame.

`db_helpers.py`

The three `UserSettings` accessors described in section 04. Anything that reaches into that table by hand is a candidate for replacement.

ONLINE, OFF GRID, AND EVERYTHING CACHED

Sentinel is built to run disconnected. Every domain has two source slots — an online URL and an off-grid one — and a two-level switch decides which is primary. Resolution is centralised in a single function so no router invents its own rule.

RESOLVE_DOMAIN_URLS() — THE ONE RESOLVER

`resolve_domain_urls(domain, db, online_default)` returns a `(primary, fallback)` tuple. It reads every setting row for the domain plus `app.connectivityMode` in one query, then: `utils.py:86–142`

1. If `{domain}.sourceOverride` is `online` or `offgrid`, that is the effective mode.
2. Otherwise (`auto`, or unset) the effective mode is the global `app.connectivityMode`, defaulting to `online`.
3. In `online` mode: primary = the online URL, fallback = the off-grid one. In `offgrid` mode the pair is reversed. The fallback is always the *other* slot — never nothing.

Two details are easy to trip over. The air domain uses different key names from every other domain (`onlineDataSourceURL` / `offgridDataSourceURL` rather than `onlineUrl` / `offgridSource`), handled by a lookup map. And an off-grid source is stored by the settings panel as an object `{"url": "…"}`, so the resolver unwraps a dict before validating. Placeholder values — `""`, `https://`, `http://localhost` — are treated as unset, and trailing slashes are stripped.

THE ADS-B CACHE LADDER

The proxy at `GET /api/air/adsb/point/{lat}/{lon}/{radius}` is the most defended endpoint in the backend, and it announces which branch it took on every response. `routers/air.py:72–212`

X-CACHE	CONDITION
`HIT`	A row exists, is within `adsb_ttl_ms`, *and* a primary URL is configured. Returned without touching the network.
`MISS`	No usable row — an upstream was called successfully, the row upserted, fresh data returned.
`BYPASS`	Upstream succeeded but the cache commit hit a lock beyond `busy_timeout`. The fresh data is served anyway rather than 500-ing.
`RATED`	The upstream answered 429. Whatever is cached is served regardless of age.
`THROTTLED`	Sentinel's *own* limiter declined the call. A refresh is at most one interval away, so the cached row is served rather than reporting an outage the upstream is not having.
`STALE`	All upstreams failed for other reasons, but the row is still within `adsb_stale_ms`.
503	No usable cache and nothing to fetch — including off-grid mode with no off-grid source configured, where the fresh-cache branch is deliberately skipped so the caller gets an honest error instead of stale data.

OUTBOUND RATE LIMITING

`MinimumIntervalRateLimiter` is a process-wide gate keyed **per upstream host**, so a local off-grid receiver is never throttled by traffic to a public API. Each waiter reserves the next free slot under a lock, then sleeps *outside* the lock — holding it while waiting would serialise the bookkeeping behind the wait and break the max-wait check. If the next slot is further away than `adsb_rate_limit_max_wait_ms`, no slot is reserved and `UpstreamThrottledError` is raised, which the router reads as "serve cache", not "upstream is down". `services/upstream_rate_limit.py:25–93`

On a 429, `penalize()` pushes the host's next free slot out by its `Retry-After` when that header is a plain number of seconds, else by `adsb_rate_limit_penalty_ms`, clamped to `adsb_rate_limit_max_penalty_ms`. A shorter penalty landing during a longer one is ignored rather than shortening it. `services/adsb.py:14–90`

> **WHY BOTH LAYERS EXIST** — the cache is keyed per lat/lon/radius, so several map panes, several tabs, or two keys expiring in the same instant can still fire a burst inside the forbidden window. The limiter is the last line of defence: it caps the wire rate no matter how many callers there are or how the cache is keyed.

A 429 is logged at `WARNING` on purpose. The local limiter is *meant* to keep the service under the threshold, so seeing one means the threshold moved and the configured interval may need raising. Non-429 upstream errors are logged with the host and status too, because otherwise a closed-off feed is indistinguishable from "no aircraft overhead" once the next source returns an empty list.

AIR, SPACE & LAND

Routers shape HTTP; services do the work. Fifteen modules under `backend/services/`, each owning one concern and importable without a request context so background tasks can call them directly.

AIR — FLIGHT HISTORY

After every successful ADS-B fetch the router queues `record_aircraft_batch()` as a FastAPI `BackgroundTask`, opening its own session so it outlives the response. Recording is opt-in: the `air.replayEnabled` preference defaults to off, and when off live data is still served and cached but the history tables are never populated. Three constants define the shape of the data: `services/flight_history.py:12–14`

`FLIGHT_GAP_MS`	10 minutes — an aircraft absent longer than this starts a new `AirFlight` row rather than extending the old one.
`SNAPSHOT_INTERVAL_MS`	10 seconds — the floor between two stored positions for the same aircraft, so a fast poll cannot inflate the table.
`MAX_HISTORY_DAYS`	30 — anything older is deleted by `cleanup_old_snapshots()`, run at startup and every 24 h thereafter.

Aircraft identity is the registration (`r`) where the feed provides one and the ICAO hex otherwise, so history survives an aircraft reappearing weeks later. The replay endpoints read back through the five indexes created at startup: available dates, a snapshot window, the aircraft list, the flights for one registration, and the snapshots of one flight.

SPACE — THE TLE PIPELINE

Every TLE, whatever its origin — Celestrak auto-fetch, a user-supplied URL, a pasted block, an uploaded file — flows through one function, `store_tle_bulk()`, which validates, deduplicates and upserts into both `tle_cache` (orbital data) and `satellite_catalogue` (identity). There is no second write path. `services/tle.py:160`

Category is never downgraded

Categories carry both a value and a source. Category specificity always beats source authority: a stored `cubesat` is never replaced by an incoming `active` or `unknown`, regardless of who says so. Only when two categories are equally specific does source priority (`celestrak_group` > `user` > `inferred` > `active`) act as tiebreaker, and only a strictly higher source wins. `services/tle.py:37–86`

A `name_source` of `user` locks a name against later TLE updates.

Radio metadata outlives a clear

Uplink/downlink/beacon/CTCSS data lives as columns on the catalogue for fast reads — but those rows are wiped by "clear TLE data". So the authoritative copy is a single `UserSettings` row (`space.satelliteRadio`), which the clear never touches, and which is re-applied onto the catalogue on every TLE import. `services/sat_radio.py`

Propagation

`services/satellite.py` wraps the `sgp4` library and outputs `[longitude, latitude]` throughout, per the GeoJSON convention. It computes position, ground track, look angles from an observer, passes, and the visibility footprint. `services/daynight.py` computes the solar terminator as a GeoJSON MultiPolygon covering the night side using nothing but `math` and `datetime` — no extra dependency. `services/satellite.py:62–402 · services/daynight.py:40`

LAND — APRS STATIONS

The Direwolf sidecar parses received packets and POSTs them in; `aprs_store` keeps one row per callsign, newest fix wins, holding position, symbol, comment, course, speed, altitude, digipeater path and the raw TNC2 packet the fix was parsed from. Retention comes from `land/aprsRetentionMinutes`, falling back to `aprs_station_ttl_ms` (5 min), and expired stations are swept by the same daily cleanup loop that prunes flight history. Like `flight_history`, it opens its own sessions so the ingest endpoint and the background sweep can both call it. `services/aprs_store.py:59–171`

TRANSPORT, FAN-OUT & SPECTRUM

Each radio is a remote `rtl_tcp` daemon reached over a raw asyncio TCP socket. The protocol is trivially simple — a 5-byte command frame, then an endless stream of unsigned 8-bit IQ pairs — and every hard problem in this module comes from the fact that an `rtl_tcp` dongle serves exactly one client.

THE COMMAND FRAME

Five bytes: one command byte, then a big-endian `uint32`. `services/sdr.py:9–16`

BYTE	COMMAND	VALUE
`0x01`	Set centre frequency	Hz
`0x02`	Set sample rate	Hz
`0x03`	Set gain mode	0 = auto, 1 = manual
`0x04`	Set gain	Tenths of a dB — 300 = 30.0 dB
`0x05`	Frequency correction	ppm
`0x08`	AGC mode	0 = off, 1 = on

ONE BROADCASTER PER RADIO

Two WebSocket handlers sharing a `StreamReader` produce `readexactly() called while another coroutine is already waiting`. So there is exactly one reader: a `RadioBroadcaster` task per radio that reads IQ and fans it out to two lists of bounded queues — computed FFT frames to spectrum subscribers, raw IQ bytes to IQ subscribers (which is also how recording works: a subscriber queue drained to a file). Queues are `maxsize=4`, so a slow client drops frames rather than stalling the reader. `services/sdr.py:692–790`

Four constants that were each a bug once

CONSTANT	WHY IT IS WHAT IT IS
`READ_CHUNK_TARGET_MS = 40`	Reads are sized **by time, not sample count**. A fixed sample count made the frame rate collapse whenever bandwidth changed — 500 kHz BW snaps the sample rate down, so an 87 040-sample read spanned ~290 ms instead of ~42 ms, giving ~3 fps and a jumpy waterfall. 40 ms ≈ 25 fps, matching the client's cap. Bounded by `READ_CHUNK_MIN_SAMPLES 4096` and `MAX 131072`.
`MAX_FFT_SIZE = 32768`	Clamped with `MIN_FFT_SIZE = 1024`. 32 k keeps the waterfall crisp to ~16× zoom on a 2000 px canvas; 65 536 was tried and the Pi could not keep up — frames dropped and audio chopped on wide bandwidths.
`RECONNECT_BACKOFF 1→10 s`	A single-client dongle shared by two Sentinel instances (or briefly stolen by a reachability probe) drops the reader. Killing the stream on first drop left the status dot stuck red; instead the broadcaster retries with capped exponential backoff for as long as clients still want the stream.
`IDLE_RELEASE_GRACE_S = 10`	After the last subscriber leaves, the socket stays open briefly to absorb a quick reconnect, then is released — so a second instance, or the reachability probe, can reach the dongle instead of being locked out by a broadcaster streaming to nobody.

> **TCP KEEPALIVE IS NOT OPTIONAL** — a rebooted radio leaves a half-open socket: the peer is gone but the connection reads `ESTABLISHED`, so a blocked `readexactly` waits the full ~10 s read timeout while the UI still shows green. Aggressive keepalive (~2 s idle, 1 s probes, dead after 3) fails the socket in about five seconds. The per-option settings are Linux-only and each is guarded, so development on macOS still gets plain `SO_KEEPALIVE`. `services/sdr.py:86–120`

SPECTRUM FRAMES

`compute_fft_frame()` uses Welch-style averaging: the IQ chunk is split into as many length-`n_fft` segments as fit, each Hann-windowed and FFT'd, and the per-bin powers averaged. Averaging N segments drops noise variance by ~1/N (≈ $10 \cdot \log_{10}(N)$ dB of smoothing) while leaving real signals where they are — the difference between a grainy noise floor and a smooth band.

Power is converted to dBFS anchored at the ADC ceiling: with IQ normalised to ±1.0 and a Hann window (coherent gain 0.5), a full-scale tone gives a bin magnitude of N/4, so $10 \cdot \log_{10}(N^2/16)$ is subtracted. That is what makes Sentinel's numbers comparable with SDR#, SDR++ and GQRX rather than being arbitrary. Bins are rounded to one decimal place before serialising.

`services/sdr.py:981–1019`

SHARED TUNING & THE WEBSOCKET PROTOCOL

One dongle, one tuner, potentially several Sentinel instances watching it. The backend resolves that with an ownership token held over a separate control channel on the fan-out relay, and mirrors the owner's state to every follower so a read-only watcher sees — and hears — exactly what the owner does.

THE OWNERSHIP MODEL

A radio's control port is its IQ port plus `sdr_relay_control_port_offset` (default 2), so IQ 1234 → control 1236. `RelayControlClient` connects there, claims the single tuning token, and runs a read loop that keeps `is_owner` and the mirrored tuner state in step with the relay's `state` pushes. Tuning goes out as semantic `set` messages: the relay is the sole writer of commands to the dongle while a token is held, applies them, and broadcasts the result — so every follower stays truthful rather than guessing. `services/sdr.py:122–175`

Three flags describe the situation, and the UI needs all three:

`CONTROL_AVAILABLE`	Whether the control channel exists at all. A raw `rtl_tcp`, or a relay without the channel, simply fails to connect and leaves this `False` — tuning then falls back to direct last-writer-wins commands over the IQ socket.
`IS_OWNER`	This instance holds the token and may retune.
`TUNER_LOCKED`	The token is held by *some* instance. This is what lets a follower distinguish "another instance is tuning" (read-only) from "the token is free" (a tune attempt can take it).

A follower that tries to retune while the token is held gets `ReadOnlyTuningError` rather than silently moving nothing, so the WebSocket layer can tell the browser it is read-only. Beyond hardware parameters the relay also carries the owner's *demod* state — NCO offset within the band, mode, audio bandwidth — and its sweep state, so a follower reproduces the exact channel the owner hears and renders the same scan/search overlay. If a relay drops those fields, followers degrade cleanly to centre-only viewing.

THE FOUR WEBSOCKETS

PATH	CARRIES
`/ws/sdr/{radio_id}`	JSON spectrum, status and control frames; accepts tuning commands. The main panel socket.
`/ws/sdr/{radio_id}/iq`	Binary IQ. Little-endian header: `uint32` sample rate, `uint32` centre Hz, then `uint8` I/Q pairs. Clients decode `(byte - 127.5) / 127.5`.
`/ws/sdr/{radio_id}/decode`	Decoded digital-voice events from the dsd-fme bridge.
`/ws/sdr/{radio_id}/decode/audio`	Decoded voice audio returned by the sidecar.

Server → client frames

TYPE	FIELDS
`spectrum`	`center_hz, sample_rate, bins, timestamp_ms`
`status`	`connected, radio_id, radio_name, center_hz, sample_rate, mode, gain_db, gain_auto, offset_hz, bw_hz, is_owner, control_available, locked`, plus the scan/search fields. Sent immediately on connect with `connected: false` — the flag only becomes true once a spectrum frame proves data is actually flowing from the physical device.
`control`	Ownership changed: another instance took or released the tuner, or this client's retune was refused. Mirrors the owner's tuning, demod and sweep state so followers react instantly instead of inferring it from the next relabelled spectrum frame.
`error`	`code, message` — `NOT_FOUND` for an unknown radio id, `CONNECT_FAILED` after three connect attempts one second apart.

Client → server commands

`tune`, `mode`, `gain` (null for auto), `squelch`, `sample_rate`, `fft_size` and `ping` — plus four that exist purely to keep followers honest: `release`, `claim`, `demod` (the owner's within-band offset, mode and bandwidth) and `sweep_state` (its scanner progress). `fft_size` is shared across subscribers — last writer wins. `routers/sdr.py:1113–1150`

> **CONNECTION IS BEST-EFFORT, THREE TIMES** — `_resolve_broadcaster()` looks the radio up, then retries `get_or_create_broadcaster` up to three times with a one-second gap before giving up. Both failure paths send an error frame and close, each wrapped so a client that has already gone away cannot raise on the way out. `routers/sdr.py:1027–1078`

SERVER-SIDE DEMOD & THE SIDECARS

Digital voice (P25, DMR, NXDN, D-STAR, YSF) and APRS are decoded by external containers, not in the browser. Both want the same thing — FM-demodulated 48 kHz mono s16 PCM over TCP — and neither can open its own connection to the dongle, because `rtl_tcp` serves one client and the backend already is that client. So the backend reproduces the browser's demodulation chain server-side and serves the result.

THE CHAIN

```
IQ decimate  →  NCO mix  →  channel LPF  →  FM discriminator  →  resample to 48 kHz
```

This is the same chain the browser AudioWorklet runs in `frontend/vue/src/composables/useSdrAudio.ts`, deliberately mirrored so server and client hear the same audio. IQ is decimated to ≤ ~1.024 Msps before the FIR, because the filter is O(taps·N) and the full stream would swamp a Pi — a 12.5 kHz channel survives the decimation intact. Output is fixed at 48 kHz mono s16, the rate dsd-fme's `-i tcp` input expects. `services/sdr_decode.py`

BRIDGES

`PcmDecodeBridge` is the shared base: it subscribes to the existing `RadioBroadcaster` IQ fan-out — never a second socket to the dongle — demodulates, and serves the PCM on a TCP port the sidecar connects to. Two subclasses specialise it: `DigitalDecodeBridge` (voice, port 7355, 12.5 kHz default) and `AprsDecodeBridge` (port 7357, 15 kHz default). Separate ports are what allow voice and APRS to decode concurrently on two dongles. Decoded voice audio returns over UDP on 7356; its true rate is *measured* on the backend and resampled to a uniform 48 kHz rather than assumed per mode.
`services/sdr_decode.py:359, 579, 723`

INGEST AUTHENTICATION

Both ingest endpoints and both config endpoints are authenticated with a shared secret in an `X-Decode-Secret` header, compared with `secrets.compare_digest`. All four **fail closed**: with no secret resolvable the endpoint answers 503 and ingestion is disabled entirely, rather than falling open. An invalid secret is 401; a valid secret with no active decode session is 409. `routers/sdr.py:1349–1400, 1463–1506`

The secret needs no configuration. `resolve_ingest_secret()` prefers an explicit `decoder_ingest_secret` setting; otherwise it reads `/run/decoder/secret`, generating a `token_urlsafe(32)` and writing it there if absent. An unwritable path falls back to an ephemeral in-memory secret, and the value is cached for the process's lifetime.

> **WHY THE FILE IS 0644** — the decoder container runs as a different uid and must read the shared volume. The secret guards the network ingest path, not local file access inside an already-trusted compose stack. `services/sdr_decode.py:83–88`

The backend stays decoder-agnostic: ingest validates JSON and relays it to the active bridge's event subscribers, defaulting a `type` of `decode_event` that the decoder may override. It routes to the single active session, so the sidecar never needs to know which radio is selected.

THE FOUR SIDECARS

CONTAINER	PROFILE	ROLE
`app`	*default*	The backend. Publishes `8080:8000`, mounts the DB volume, the backend source (live reload), map assets, the built SPA read-only and the shared decoder-secret volume. Its `host.docker.internal:host-gateway` entry exists because Sentry runs with `network_mode: host` and has no container DNS name — `localhost` in a radio address would resolve to this container instead.
`decoder`	`decoder`	dsd-fme digital voice. Opt-in because building it compiles the patent-encumbered mbelib vocoder locally; the image never leaves your machine, and it is never built in CI.
`aprs-decoder`	`aprs`	Direwolf. Plain GPL, but still gated so it only runs when APRS is wanted. Independent of the voice decoder — both profiles can run together.
`adsb-decoder`	`adsb`	readsb, decoding 1090 MHz from a *remote* Sentry dongle and publishing `aircraft.json` for the Off Grid air source. It polls `/api/sdr/adsb/config`, so changing the source device takes effect at the next reconnect.

A CLIENT, NOT A SECOND OWNER

Sentry is a separate product — a Raspberry Pi running SDR dongles behind its own management API. ADR-0009 settles the boundary: **Sentry owns device state; Sentinel is a full remote client of its existing `/api` surface.** Sentinel reimplements none of Sentry's validation and persists none of its device state.

WHAT SENTINEL STORES, AND WHAT IT REFUSES TO

The `sentry_hosts` table holds only what is genuinely Sentinel's own: how to reach one host (address, port, bearer token), whether the poller should poll it, and a little telemetry — `last_seen_at` and `last_error` — so the UI can show reachability without polling itself. Device state is cached in memory by the fleet poller and always read live through the client. The auth token is write-only over the API. `models.py:292–314`

THE CLIENT

`SentryClient` is built per host from the row and does one job: shape the HTTP call and translate the response into a value or one of two typed errors. That distinction is the point — `SentryUnreachableError` means no HTTP response ever arrived (DNS, connect, timeout), so there is nothing of Sentry's to surface; `SentryApiError` carries Sentry's own `{code, message}` envelope verbatim, because ADR-0009 requires Sentinel to surface Sentry's rejection rather than inventing its own. A caller can always tell "the Pi was off" from "Sentry said no". `services/sentry_client.py:104–137`

> **ADDRESS VALIDATION IS AN ALLOW-LIST** — `address` is operator input used to build a URL, so it is validated before it can reach httpx: rejected outright if it contains `://`, `@`, `/` or whitespace; rejected if it contains `:` (no port suffix, no IPv6 literal); capped at 253 characters; then it must parse as an IPv4 literal or match a strict RFC-1123 hostname pattern. The port is validated separately to 1–65535. Rejection messages are operator-facing and surfaced directly as 422. `services/sentry_client.py:66–102`

THE FLEET POLLER

One asyncio task per **enabled** host polls `GET /api/status` every `sentry_poll_interval_s` (2 s) and caches the snapshot in memory, so a router read never blocks on a round-trip to a Pi that may be slow or absent. On failure it backs off exponentially from 2 s to a 30 s cap — the same shape as the SDR broadcaster's reconnect — so a long-dead host is retried periodically rather than hammered or abandoned. `services/sentry_fleet.py:49–130`

Each host has a refresh `Event`: after a write (device PATCH or DELETE, serial flash, hotspot mutation) the router calls `refresh_now()` to wake the poller immediately, so the cached snapshot cannot sit stale for a full interval after a change the user just made. `start_all()` runs at startup, `stop_all()` in shutdown, and `restart_host()` after an edit to address, port, token or enabled flag. A single module-scope `fleet_poller` is the instance the app uses; the class is not a singleton so tests can build their own.

CLAIMING A DONGLE FOR ADS-B (ADR-0003)

Off Grid air data used to be a URL and nothing else, which left the receiver anonymous: Sentinel knew where to *read* aircraft from but not which dongle produced them, so it could neither tune it nor stop anything else retuning it — the map went empty and said nothing about why. The `air.offgridSdrSource` preference now names a `{sentry_host_id, sentry_device_id}` pair, and `claim_and_tune()` resolves it to a live device, claims it and tunes it to 1090 MHz **in one call**. Tuning travels with the claim deliberately: two calls would leave a window where the device is ours but still on the wrong frequency, and a second request that can fail on its own means handling a "claimed but not tuned" state that nothing wants to own. `services/adsb_source.py:1–221`

WHAT IS DEFENDED, AND HOW

Sentinel is a single-tenant appliance: it has no user accounts, no sessions and no authorization model, because there is one operator and the service is not meant to face the public internet. That makes the threat model narrow but specific — hostile *input*, hostile *upstreams*, and anything on the LAN that can reach an ingest port.

SURFACE	CONTROL
SQL injection	Structurally prevented: every query uses the SQLAlchemy ORM or bound `:params`. There is no string-interpolated SQL anywhere in the backend, including the startup migrations, which use fixed literals and bound values.
Request bodies	Pydantic models at the edge of every write endpoint; FastAPI answers 422 before a handler runs. Path and query parameters are typed, so `lat` / `lon` / `radius` cannot arrive as text.
Free-text names	`clean_group_name()` strips C0/C1 controls, zero-width characters and bidi overrides, normalises NFKC, collapses whitespace, and enforces a 60-character cap — guarding the risks the ORM does not: corrupted JSON config, forged log lines and spoofed UI text.
Operator-supplied hosts	Sentry addresses pass a strict allow-list before reaching httpx — no scheme, credentials, path, whitespace, port suffix or IPv6 literal (section 11). Ports are range-checked.
Coordinates	`_validated_location()` accepts either a fully-empty pair (meaning "use browser geolocation") or a valid in-range float pair, and rejects partials outright — on the single-setting PUT and on config upload, where it is checked *before* anything is written so a bad value cannot be half-persisted.
Sidecar ingest	Shared secret in `X-Decode-Secret`, constant-time compared, failing closed to 503 when no secret is resolvable. Auto-generated as `token_urlsafe(32)` into a volume both containers mount, so it is never a committed value and never a default.
Hostile upstream headers	An upstream `Retry-After` is honoured only when it parses as a number of seconds, and is clamped to `adsb_rate_limit_max_penalty_ms` (10 min) — a misconfigured or malicious header cannot park the feed for hours.
Error responses	Handlers return `{"error": …}` shapes rather than tracebacks; unexpected exceptions become a 500 with the exception string, and upstream detail is logged rather than returned. Sentry's errors are the deliberate exception — its envelope is passed through verbatim by design.
Secrets in the repo	None. `decoder_ingest_secret` defaults empty and auto-generates; compose reads `SENTINEL_DECODER_SECRET` from a git-ignored `.env`; the Sentry bearer token lives only in the database and is write-only over the API.

KNOWN LIMITS — STATED, NOT HIDDEN

- **No authentication on the API.** Anyone who can reach port 8080 can read and write every setting and command every radio. The deployment assumption is a trusted LAN.
- **No CORS configuration.** None is needed while the SPA is served same-origin by the same process; exposing the API to another origin would require adding it deliberately.
- **No rate limiting on inbound requests.** The limiter is outbound-only.
- **The IQ stream is unencrypted** across the LAN, as is `rtl_tcp` itself — the protocol has no security model.
- **The ingest secret file is world-readable** inside the compose stack, deliberately (section 10).
- **SQLite is not hardened against a hostile local user** — it is a file in a volume.

> **IF THIS EVER FACES THE INTERNET** — authentication, inbound rate limiting and TLS termination all have to be added first. Nothing in the current design assumes an untrusted caller.

KEEPING IT HONEST

THE TEST HARNESS

783 tests across 25 files under `tests/backend/`. One root `conftest.py` supplies the fixtures every router test uses:

FIXTURE	WHAT IT DOES
`test_engine`	Per-test in-memory SQLite. `StaticPool` is required — without it each new connection gets its own empty in-memory database.
`db_setup`	Creates the schema from the live ORM metadata, so a model change is reflected in tests automatically, then disposes the engine.
`client`	`TestClient` with `get_db` overridden. Deliberately *not* used as a context manager: entering it would trigger the lifespan, which starts a 24-hour background task and creates the real production schema. Router tests do not need the lifespan.
`isolate_sdr_data_files`	Autouse. Redirects `FREQUENCIES_FILE` and `BANDPLAN_FILE` into a tmp dir, because frequency CRUD writes the committed seed files back to disk — without it a test run overwrites the repo's curated data with whatever the in-memory DB held.

> **RUN FROM THE REPO ROOT, WITH THE BACKEND PROJECT** — `uv run --project backend pytest`. The `pyproject.toml` lives in `backend/`, so a bare `uv run` from the root resolves the wrong interpreter. `pytest.ini` declares `required_plugins = pytest-asyncio` as a tripwire: without it a wrong interpreter silently ignores `asyncio_mode` and every async test fails with confusing 500s.

GATES

COMMAND	GATING?	NOTES
`uv run --project backend pytest`	**Yes**	All 783.
`ruff check backend`	**Yes**	Deliberately minimal ruleset — `E, F, I, UP, B` — with `E501`, `B008` (FastAPI's `Depends()` is explicit by design), `E701` and `UP047` ignored.
`ruff format --check backend`	**Yes**	The single source of Python formatting truth: double quotes, spaces, magic trailing commas honoured, 120 columns.
`mypy backend`	No	Informational. ~90 errors, nearly all SQLAlchemy `Column[T]` vs unwrapped values on attribute access; clearing them means annotating every model with `Mapped[T]`. Wired up, not enforced.

ADDING TO THE BACKEND — THE SHORT VERSION

- → **A new endpoint** on an existing resource: add it to that router, with a Pydantic body model, `Depends(get_db)`, an explicit status code and a docstring naming its error codes. Update the module docstring's endpoint list.
- → **A new resource**: a new router module with its own prefix and tag, included in `main.py`. Never bolt an unrelated concern onto an existing router.
- → **A new column**: add it to the model *and* append a guarded `ALTER TABLE` in `create_tables()`. Backfill existing rows in the same place if the column is non-nullable in spirit.
- → **A new preference**: add it to `default_config.json` and read it with `get_setting()` — no migration, no model change. Removing one later means adding its `(namespace, key)` to `_REMOVED_SETTING_KEYS`.
- → **A new upstream call**: put it behind a service module, route it through the rate limiter, and give it a cache with a fresh TTL and a stale window. Answer from cache on failure, and label how.
- → **A new map feature** is a *frontend* change — a MapLibre `IControl`, not a backend endpoint. Check whether the data you need is already on the wire before adding a route.

WHERE THE DECISIONS ARE WRITTEN DOWN

Four architecture decision records under `docs/adr/`: **0001** the webapp-standards retrofit, **0002** base-component dedupe, **0003** the Sentry SDR lock-and-tune contract behind the Off Grid ADS-B source, and **0004** the removal of trunk tracking. Where this guide and an ADR disagree, the ADR states the intent and the code states the fact.